DSA4212:
Gaussian Processes

August 27, 2025

Group Composition:

1. Albin Erik Liljefors, A0307006U, e1384720@u.nus.edu
2. Tiziano Di Gregorio Botek, A0293650B, e1345421@u.nus.edu
3. Jonathan Hvitved, A0293747M, e1345518@u.nus.edu
4. Alejandro Trigueros Alonso, A0293701H, e1345472@u.nus.edu

Contents

1 Introduction **3**

2 Theory **3**

2.1 Introduction to Gaussian Processes . . . 3
2.1.1 Kernel Functions . . . 4
2.1.2 Kernel Selection and Tuning Hyperparameters . . . 5
2.2 Mathematical Formulation . . . 5
2.2.1 Prior and Posterior . . . 6
2.2.2 Inference in GPs . . . 6
2.3 Challenges with Large-Scale Datasets . . . 6
2.3.1 Computational Complexity . . . 6
2.4 Scalable Approaches for Gaussian Process Regression . . . 7
2.4.1 Sparse Gaussian Processes . . . 7
2.5 Uncertainty Quantification in Gaussian Processes . . . 7

3 Method **8**

3.1 Data Preparation . . . 8

4 Results and Discussion **9**
4.1 Predictive Performance and Accuracy 9
4.2 Uncertainty Quantification and Calibration 10
4.3 Computational Efficiency 11
4.4 Error-Uncertainty Relationship 11
4.5 Implementation Limitations 12

5 Conclusion **12**

1 Introduction

In supervised machine learning, accurately modeling complex, uncertain relationships between inputs and outputs is crucial, particularly when the available data is sparse or noisy. Gaussian Process (GP) regression offers a powerful, non-parametric approach to this challenge, providing not only predictions but also an estimate of uncertainty around these predictions. This probabilistic framework has become especially valuable in applications where reliable uncertainty quantification is essential, such as robotics, environmental modeling, and financial forecasting. However, despite its advantages, the computational demands of GP regression scale poorly with large datasets, presenting significant challenges for its practical use in high-dimensional or extensive data scenarios. In this project, we aim to explore various GP implementations, including scalable alternatives like Sparse Gaussian Processes, to assess their performance in terms of predictive accuracy, computational efficiency, and uncertainty estimation.

2 Theory

2.1 Introduction to Gaussian Processes

To model the objective function in supervised machine learning, one can consider an infinite set of functions, $f(x) \in S$, which make predictions for the input x. By assigning a prior probability to every function in S, functions with a higher probability are considered to be more likely and thus will model the objective function better.

Because the set of functions is infinite, some selection process it needed to model the objective function in finite time. A Gaussian Process is a generalization of the Gaussian probability distribution, which governs the properties of functions. If one only consider the properties of the objective function at a finite number of points, then inference in a Gaussian process will yield the same answer. [RW05].

The following defines a Guassian Process, taken from [Dud10][p.443]
Definition 3.1 (Gaussian Process) *Let X be a non-empty set, $k : \mathcal{X} \times : \mathcal{X} \to \mathbb{R}$ be a positive definite kernel and $m : \mathcal{X} \to \mathbb{R}$ be any real-valued function. Then a random function f: $X \to \mathbb{R}$ is said to be a Gaussian Process (GP) with mean function m and covariance kernel k, denoted $\mathcal{GP}(m, k)$, if the following holds: For any finite set $X = (x_1, \ldots, x_n) \in \mathcal{X}$ of any size $n \in \mathbb{N}$, the random vector*

$$f_X = (f(x_1), \ldots, f(x_n))^T \in \mathcal{R}^n$$

follows the multivariate normal distribution $\mathcal{N}(m_X, k_{XX})$ with covariance matrix $k_{XX} = (k(x_i, x_j))_{i,j=1}^n \in \mathbb{R}^{n \times x}$ and mean vector $m_X = (m(x_1), \ldots, m(x_n))^T$

If we consider a regression problem, where (X, Y) is an training input-output pair, the Gaussian Process regression model is given by

$$y_i = f(x_i) + \epsilon_i \tag{1}$$

with noise $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ and the prior distribution over $f \sim \mathcal{GP}(\mu, k)$.

We will from now assume that the mean vector of the GP is zero, $\mu = 0$, which is a common approach in Gaussian Process Regression. This makes the prior distribution more simple, and the GPs are still flexible enough to model the posterior distribution. [Kra18]

If (X^*, Y^*) is a test input-output pair then by combining the data into a single vector yields

$$\begin{bmatrix} Y \\ Y^* \end{bmatrix} = \begin{bmatrix} f(X) \\ f(X^*) \end{bmatrix} + \begin{bmatrix} \epsilon \\ \epsilon^* \end{bmatrix} \sim \mathcal{N}\big(0, \begin{bmatrix} k(X,X) & k(X,X^* \\ k(X^*,X) & k(X^*,X^*) \end{bmatrix}\big) + \mathcal{N}\big(0, \begin{bmatrix} \sigma^2 I & \\ & \sigma^2 I \end{bmatrix}\big) \tag{2}$$

$$= \mathcal{N}\big(0, \begin{bmatrix} k(X,X) + \sigma^2 I & k(X,X^* \\ k(X^*,X) & k(X^*,X^*) + \sigma^2 I \end{bmatrix}\big) \tag{3}$$

Conditioning the probability of observing Y^* given X^* and the prior distribution yields the posterior predicative probability distribution,

$$P(Y^*|Y, X, X^*) \sim N(\mu, \Sigma) \tag{4}$$

Where μ is the mean and Σ is the covariance matrix [Pry24].

$$\mu = k(X^*)(k(X,X) + \sigma^2 I)^{-1} Y \tag{5}$$

$$\Sigma = k(X^*, X^*) + \sigma I - k(X^*, X)(k(X,X) + \sigma^2 I)^{-1} k(X, X^*) \tag{6}$$

2.1.1 Kernel Functions

In Gaussian Process Regression (GPR), the kernel function $k(x_i, x_j)$ plays a central role in defining the covariance between two points x_i and x_j, and thereby controls the generalization properties of the model. A kernel function $k(x_i, x_j)$ must be positive definite, ensuring that for any set of inputs $\{x_1, x_2, \ldots, x_n\}$, the corresponding Gram matrix K, where $K_{ij} = k(x_i, x_j)$, is positive definite. This ensures that the covariance matrix is invertible, which is essential for the Gaussian Process framework.

The kernel function typically contains several hyperparameters, which control its behavior and, by extension, the properties of the Gaussian Process. Hyperparameters represent degrees of freedom in the model, allowing it to adapt to different data characteristics. These hyperparameters directly influence how the kernel evaluates the similarity between input points, thereby shaping the predictions.

Formally, if we denote a kernel function by $k(x_i, x_j; \theta)$, where $\theta = (\theta_1, \theta_2, \ldots, \theta_m)$ represents the set of hyperparameters, then the structure of the covariance matrix K and thus the prior distribution over functions is governed by these parameters.[Bec21]

To illustrate how these hyperparameters function in practice, we consider a commonly used kernel function:

Squared Exponential (RBF) Kernel:

$$k(x_i, x_j) = \sigma_f^2 \exp\left(-\frac{\|x_i - x_j\|^2}{2l^2}\right),$$

where l is the lengthscale hyperparameter, and σ_f^2 is the signal variance. Here, the lengthscale l governs how quickly the covariance decays as the distance between points x_i and x_j increases. A small l leads to rapidly decaying covariances, allowing the model to capture fine-grained details in the data, while a large l produces smoother predictions that change more gradually. The signal variance σ_f^2 dictates the magnitude of variation in the function values.

The hyperparameters, such as the lengthscale l and signal variance σ_f^2, are crucial in controlling the behavior of the model. Their values dictate how the kernel function behaves and how flexible the Gaussian Process model is in fitting the data.[Bec21]

2.1.2 Kernel Selection and Tuning Hyperparameters

Choosing the appropriate kernel and setting the corresponding hyperparameters is critical for model performance. One common approach to select hyperparameters is by maximizing the *log marginal likelihood*. This method evaluates how well the Gaussian Process model fits the data by integrating over all possible functions governed by the prior distribution. Given a training dataset $D = (X, Y)$, the log marginal likelihood $\log p(Y|X, \theta)$ is computed as:

$$\log p(Y|X, \theta) = -\frac{1}{2} Y^T (K_\theta + \sigma_n^2 I)^{-1} Y - \frac{1}{2} \log |K_\theta + \sigma_n^2 I| - \frac{n}{2} \log 2\pi,$$

where K_θ is the covariance matrix defined by the kernel with hyperparameters θ, and σ_n^2 is the noise variance. The log marginal likelihood consists of three terms:

- The first term, $-\frac{1}{2} Y^T (K_\theta + \sigma_n^2 I)^{-1} Y$, measures how well the model fits the data.
- The second term, $-\frac{1}{2} \log |K_\theta + \sigma_n^2 I|$, penalizes model complexity by discouraging overly flexible models with large covariance matrices.
- The third term, $-\frac{n}{2} \log 2\pi$, is a constant normalization factor.

Maximizing this likelihood function leads to the optimal hyperparameters by balancing data fit and model complexity.[Bec21]

2.2 Mathematical Formulation

Deriving the Gaussian Process model for regression can be done by starting with a standard linear regression model [RW05]

$$y = f(\mathbf{x}) + \epsilon, \quad f(\mathbf{x}) = \mathbf{x}^T \mathbf{w} \tag{7}$$

with $\mathbf{w}$ being the weights of the model and the noise, ϵ, following an independent, identically distributed Gaussian distribution

$$\epsilon \sim \mathcal{N}(0, \sigma_n^2) \tag{8}$$

2.2.1 Prior and Posterior

The probability density, given a set of parameters, is given by

$$p(y|X, \mathbf{w}) = \Pi_{i=1}^{n} p(y_i|\mathbf{x}_i\mathbf{w}) \cdots = \mathcal{N}(X^T\mathbf{w}, \sigma_n^2 I) \tag{9}$$

By specifying a prior over the parameters and assumption is made about their characteristics before studying the observations. One such an assumption is to put a zero mean Gaussian prior with a covariance matrix [RW05].

$$\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \Sigma) \tag{10}$$

Computed by Baye's rule, the posterior distribution of the weights are determined by the likelihood and prior distribution,

$$p(\mathbf{w}|\mathbf{y}, X) = \frac{p(\mathbf{y}, X, \mathbf{w})p(\mathbf{w})}{p(\mathbf{y}|X)}. \tag{11}$$

The posterior distribution is again Gaussian with mean $\overline{\mathbf{w}}$ and some covariance matrix A^{-1} such that

$$p(\mathbf{w}|X, \mathbf{y}) \sim \mathcal{N}(\overline{\mathbf{w}}, A^{-1}) \tag{12}$$

2.2.2 Inference in GPs

By utilizing the posterior distribution of the weights, a prediction on a test input is made by averaging over all possible parameter values. The predicative distribution $f(\mathbf{x}_*)$ is given be averaging over all possible linear models given the Gaussian posterior[RW05].

$$p(f_*, \mathbf{x}_*, X, \mathbf{y}) =\sim (\frac{1}{\sigma_n^2}\mathbf{x}_*^T A^{-1} X\mathbf{y}, \mathbf{x}_*^T A^{-1} X\mathbf{x}_*) \tag{13}$$

2.3 Challenges with Large-Scale Datasets

There are two major factors which affect the efficiency of doing predictions with GPs. The samples size, n, and dimensionality of the input, d impose both challenges with computational complexity and memory constraints.

2.3.1 Computational Complexity

The complexity of computing the inverse covariance matrix is $\mathcal{O}(n^3)$.As a result, applying GPs to large datasets becomes highly infeasible due to the excessive computational costs involved, particularly when n is large. The size of hyperparameters, θ, grows linearly with the dimensionality of the input d. Because each dimension requires its own set of hyperparameters, depending on the kernel function, the complexity of optimizing the marginal likelihood during training is further increased for high-dimensional data. [JPN22]

2.4 Scalable Approaches for Gaussian Process Regression

Multiple approaches to apply Gaussian Process Regression (GPR) to larger datasets exists. Here we choose to focus on the method of GPR using Sparse Gaussian Processes.

2.4.1 Sparse Gaussian Processes

The main problem of the exact Gaussian process is the computational requirements needed on the training set. Its time complexity scales with a factor of $O(n^3)$ whereas the sparse one reduces it to $O(n^2m)$ with n being the size of the training set and m the number of variables included in the sparse Gaussian models ([BVdWR16]).

For the sparse Gaussian model, only a subset of all the variables is used to evaluate the inputs. hence f_m becomes an approximation of f. This gives the approximate posterior:

$$q(f_*) = \int p(f_* \mid f_m)\phi(f_m)\, df_m$$

where $\phi(f_m)$ is an approximation to the intractable $p(f_m \mid y)$

Hence, the only difference additional to find the optimal values for μ_m and A_m remains to find the optimal value of the inputs points X_m since the approximation function depends on those particular inputs.

As this method depends on how well f_m approximates f, it is important that the inducing points are selected in a way such that the final set of inducing points is a representative sample of the full set of points. Classical approaches include K-means clustering and farthest point sampling. By using such selection methods we aim to pick a subset of points that represent the original "shape" of the full dataset as well as possible, which is optimal in the method of Sparse Gaussian Processes, to make sure that the posterior distribution using the sparse method approximates the posterior distribution from the exact method as good as possible.

2.5 Uncertainty Quantification in Gaussian Processes

Gaussian Processes (GPs) hold a distinct position in uncertainty quantification (UQ) by leveraging a non-parametric, probabilistic approach, as discussed throughout our exploration of GP theory. By representing the objective function as a distribution over possible functions, GPs offer a flexible and interpretable framework for capturing both predictive mean and uncertainty. This dual capability aligns GPs closely with the core goals of UQ: to not only predict outcomes but also to provide an associated measure of confidence, or uncertainty, around those predictions.

In the broader context of UQ, GPs are particularly well-suited for scenarios where the relationship between inputs and outputs is complex and potentially nonlinear. This makes them a central tool in applications where modeling unknown functions with limited data is essential. Unlike parametric approaches, which assume a specific functional form, GPs assume a distribution over functions defined by a kernel function, allowing them to adapt to a wide range of data patterns

and scales of uncertainty. This characteristic sets them apart from other common UQ methods, like Bayesian linear regression, which can be restrictive when modeling more complex relationships.

Comparing GPs to other UQ methods, such as Monte Carlo (MC) simulations or Polynomial Chaos Expansions (PCE), helps highlight GPs' strengths and limitations. MC methods, for example, generate a large number of samples to approximate uncertainties, making them versatile but computationally intensive. GPs, on the other hand, directly model the uncertainty by assigning prior distributions over functions, which can be computationally efficient in lower-dimensional cases. However, the $\mathcal{O}(n^3)$ complexity in GPs limits their scalability for large datasets, where sparse approximations or variational methods become necessary.

Polynomial Chaos Expansions (PCE) offer an efficient approach to uncertainty quantification by expanding the output function in terms of orthogonal polynomials, enabling a more efficient estimation of output distributions based on known input distributions. This approach is particularly advantageous for complex and computationally expensive output functions. In contrast, Gaussian Processes (GPs) provide a more flexible alternative when input distributions are unknown or complex, relying on observed samples to model uncertainty through a posterior distribution in regression contexts. GPs are especially useful with limited sample data, capturing uncertainty without needing a fully defined input distribution.

In summary, GPs contribute a powerful, probabilistic method to the field of uncertainty quantification, especially valuable in applications requiring interpretable models with inherent flexibility. While GPs face computational limits with large data, their versatility and theoretical soundness make them an essential approach within UQ.

3 Method

The purpose of this comparison is to evaluate different Gaussian Process (GP) implementations and their efficiency in handling large-scale data in terms of both computational performance and predictive accuracy, measuring RMSE and uncertainty progress. All experiments were conducted using Python with libraries such as `scipy`, `jax`, and `scikit-learn`.

3.1 Data Preparation

4 Results and Discussion

Our experimental evaluation compares Gaussian Process Regression (GPR) and Sparse Gaussian Process Regression (SGPR) implementations across different optimizers (ADAM, CG, L-BFGS-B). We analyze results in terms of predictive accuracy, uncertainty quantification, computational efficiency, and error-uncertainty relationships.

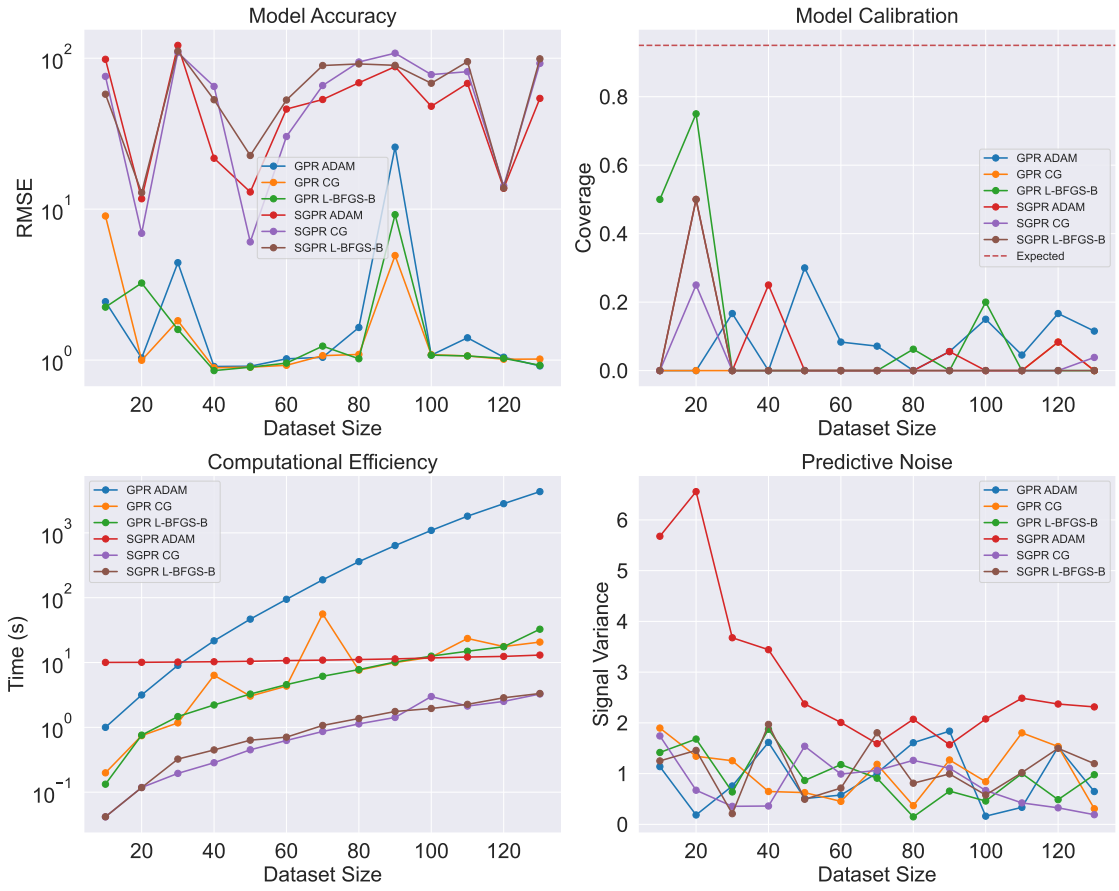


Figure 1: Performance summary of GPR and SGPR variants across different metrics: (a) Model accuracy showing RMSE vs dataset size, (b) Model calibration showing empirical coverage against expected coverage (dashed line), (c) Computational efficiency showing training time vs dataset size, and (d) Predictive noise showing signal variance vs dataset size.

4.1 Predictive Performance and Accuracy

The accuracy results (Figure 1) show:

- **GPR methods** maintain stable RMSE values around 10^0 (1-2 RMSE).

- **SGPR methods** exhibit significantly higher errors, with RMSE values reaching up to 10^2, and show high volatility.
- **Optimizer choice** has minimal impact within each method type (GPR vs SGPR).
- **Anomalous behavior** is observed around $N = 80 - 90$ for both methods.

4.2 Uncertainty Quantification and Calibration

The calibration analysis reveals severe issues across all implementations:

- All models fail to achieve the expected 95% coverage rate.
- The **best case** is 75% coverage with GPR L-BFGS-B at $N = 20$.
- Most models show **near-zero coverage**, especially as dataset size increases.

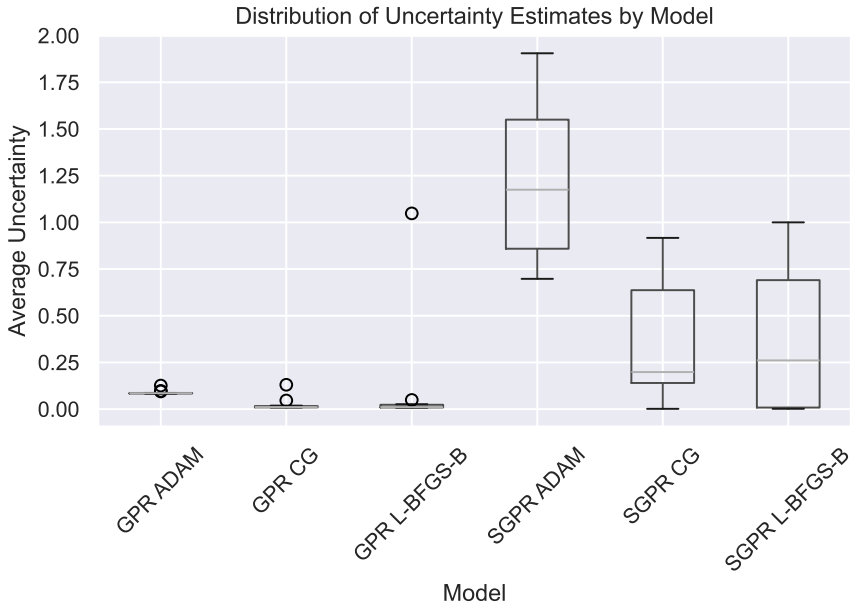


Figure 2: Distribution of uncertainty estimates across different model variants. Box plots show median, quartiles, and outliers of average uncertainty values for each model, highlighting the systematic differences between GPR and SGPR implementations.

Uncertainty distributions (Figure 2) show:

- **GPR models** provide consistent, low uncertainty estimates across optimizers.
- **SGPR models** show 5-10x higher uncertainty values with greater variability.
- **Paradoxical behavior** in SGPR is observed, where high uncertainty values fail to capture true coverage.

4.3 Computational Efficiency

Efficiency results demonstrate clear trade-offs between GPR and SGPR implementations:

- **GPR models** exhibit exponential growth in computational time, scaling from 1s to over 1000s as dataset size increases.
- **SGPR models** maintain near-constant computational time, with only linear growth.
- The efficiency gap becomes particularly pronounced for datasets larger than $N = 60$.

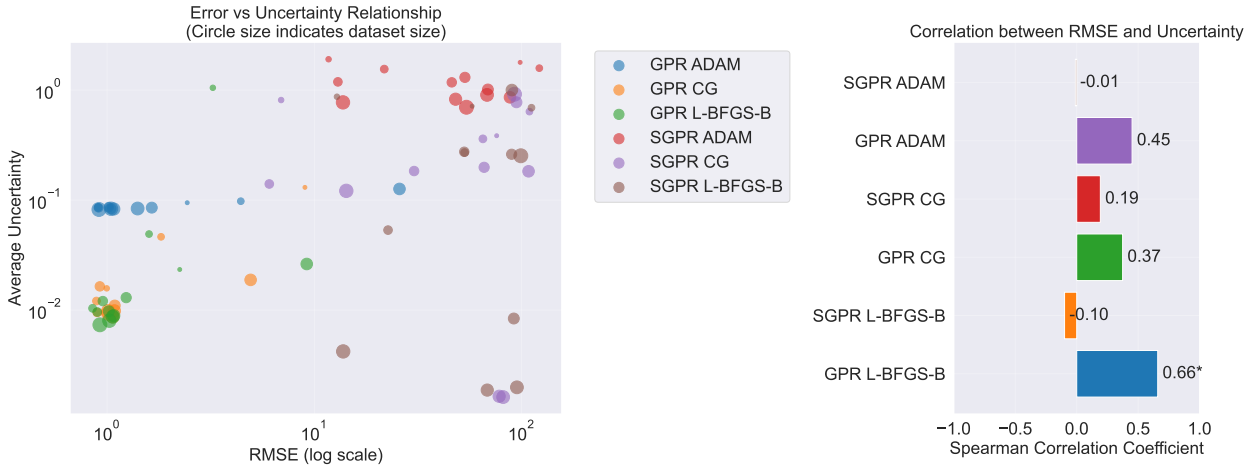


Figure 3: Error-uncertainty relationship analysis. Left: Scatter plot of average uncertainty vs RMSE (log scale) with point sizes indicating dataset size. Right: Spearman correlation coefficients between RMSE and uncertainty for each model.

4.4 Error-Uncertainty Relationship

Analysis of error-uncertainty relationships (Figure 3) reveals:

- **GPR models** show a strong positive correlation between RMSE and uncertainty, with a correlation of 0.66 for L-BFGS-B.
- **SGPR models** display weak correlations, ranging from -0.10 to 0.19, suggesting unreliability in uncertainty estimates.
- SGPR tends to produce **high uncertainties regardless of actual error**.

4.5 Implementation Limitations

Our key findings highlight several limitations in our implementations:

1. **Computational Efficiency at Cost of Accuracy:** SGPR achieves computational efficiency but at a significant cost to predictive accuracy.
2. **Severe Uncertainty Miscalibration:** All implementations show major issues in uncertainty calibration, failing to provide reliable confidence intervals.
3. **Unreliable Confidence Intervals in SGPR:** SGPR models produce high uncertainty values but fail to achieve meaningful confidence intervals.

While our implementations demonstrate the computational benefits of sparse approximation, they struggle with uncertainty quantification and accurate predictions, underscoring the importance of advanced frameworks for practical deployment.

5 Conclusion

In this study, we compared standard Gaussian Process Regression with its sparse variant across different optimizers, focusing on predictive accuracy, uncertainty quantification, and computational efficiency. Our experimental results reveal important trade-offs in GP implementations.

The standard GPR demonstrated superior predictive performance, maintaining RMSE values around 1-2, but suffered from cubic computational scaling that became prohibitive for larger datasets. In contrast, SGPR achieved the promised computational efficiency with near-constant training times, but at a significant cost to predictive accuracy, often showing RMSE values 100 times higher than standard GPR.

A particularly concerning finding was the poor uncertainty quantification across all implementations. While GPs theoretically provide well-calibrated uncertainty estimates, our implementations failed to achieve reliable coverage of the true values, with coverage rates far below the expected 95% confidence level. This was especially pronounced in SGPR, which paradoxically showed high uncertainty values while failing to capture true values within its confidence intervals.

Our study demonstrates that while sparse approximations can address the computational challenges of GPs, maintaining the probabilistic advantages of the original framework remains a significant challenge. This suggests that practical GP implementations require careful consideration of the trade-offs between computational efficiency and reliable uncertainty quantification, particularly in applications where accurate uncertainty estimates are crucial.

References

[Bec21] Thomas Beckers. *An Introduction to Gaussian Process Models*. Technical University of Munich, 2021.

[BVdWR16] Matthias Bauer, Mark Van der Wilk, and Carl Edward Rasmussen. Understanding probabilistic sparse gaussian process approximations. *Advances in neural information processing systems*, 29, 2016.

[Dud10] R. M. Dudley. *Real analysis and probability*, volume 74.;74;. Cambridge University Press, Cambridge, second;2;new;new; edition, 2002;2010;.

[JPN22] Mengrui Jiang, Giulia Pedrielli, and Szu Hui Ng. *Gaussian Processes for High-Dimensional, Large Data Sets: A Review*. IEEE, 2022.

[Kra18] Martin Krasser. Gaussian processes blog post, 2018. Accessed: 2024-11-08.

[Pry24] Rory Pryzant. Gaussian processes blog post, 2024. Accessed: 2024-10-17.

[RW05] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. The MIT Press, 11 2005.